

Tutorial AODV Lab

This lab explores Ad hoc On-demand Distance Vector routing while one relay moves out of radio range and back. You will build the supplied INET model, watch route discovery and repair in Qtenv, and analyze delivery, delay, hop count, and interruption time. The supplied AodvMobilityLab-main folder contains the source files and omnetpp.ini.

1 Prepare the workspace

Use an OMNeT++ installation with a compatible INET Framework installation. INET must be visible as an open project in the same IDE workspace. This lab uses INET MANET routers, AODV, wireless radios, PingApp, and canvas visualizers. Keep the supplied files together. Exact numeric outputs depend on your OMNeT++ and INET versions.

1. Start the OMNeT++ IDE and choose a workspace directory. If INET is not already present, install it and confirm that the inet project builds successfully.

2. Create an OMNeT++ project named AodvMobilityLab, with C++ support enabled. Copy the supplied files from AodvMobilityLab-main into the project root.

3. Confirm that the NED files, C++ files, and omnetpp.ini appear together. Do not create an extra nesting level that is absent from the configured NED source folder.

If you use a subfolder, set it as both the NED source folder and C++ source location. Point the run configuration working directory and ini file to that same folder.

4. Add the inet project reference. Under OMNeT++ project properties, include the folder containing the NED files and generate or refresh the Makefile.

5. Build the project. Build INET first if prompted, then resolve compile and NED errors before running the simulation.

2 Import an existing OMNeT++ project

Use this path only when a project directory already contains a .project file. The supplied AodvMobilityLab-main folder is a source folder without a project descriptor, so creating a project and copying its files is the reliable path. Verify the inet reference, NED source folder, C++ build configuration, and ini working directory.

3 Understand what the files build

File or group	Role in the simulation
AodvMobilityNetwork.ned	Defines the shared ten-node topology plus AODV and DSDV network variants.
omnetpp.ini	Sets protocol parameters, radio range, traffic timing, mobility, visualization, and result recording.
ManetMetrics.cc/.h/.ned	Computes destination delivery, one-way delay, hop count, PDR, and reception gap.
AnimationController and visualizer files	Control movement trails, accepted-hop arrows, labels, route overlays, and selected drops.
Makefile and project files	Build the C++ modules and connect the project to INET.

The topology is deliberately sparse. With a 250 m communication range, the initial central path can use host[0] -> host[1] -> host[2] -> host[3] -> host[9] (four links). An upper route can use host[0] -> host[4] -> host[5] -> host[6] -> host[3] -> host[9] (five links). Wireless links are determined by radio range at run time; these are geometric candidate paths, not guaranteed routing-table choices.

4 Run AODV

1. Open omnetpp.ini and select the AodvMobilityDemo configuration. Run it with QtEnv so the canvas, labels, route overlays, and packet arrows are visible.
2. Set the working directory to the folder containing omnetpp.ini. Run to the 100 s simulation limit and pause around the times in the observation table.
3. Watch host[0] (SOURCE), host[9] (DESTINATION), and the yellow host[2] MOVING RELAY. Blue RREQ, green RREP, red RERR, black PING, and red DROP labels identify different phases.

Parameter guide. sim-time-limit is 100 s, so the run covers departure and return. seed-set = 41 makes repeated launches reproducible. The radio communicationRange is 250 m; lowering it breaks links earlier, while increasing it can hide route repair. AODV activeRouteTimeout = 2 s controls how long an unused route remains active, deletePeriod = 3 s controls invalid-route retention, helloInterval = 1 s controls neighbor liveness messages, and allowedHelloLoss = 2 controls how many missed Hellos trigger failure. PingApp starts at 2 s, sends a 256 B packet every 0.6 s, and uses hopLimit = 32.

What the custom code measures

ManetMetrics subscribes to PingApp signals and destination-side packet activity. It counts delivered requests, computes one-way delay from packet creation, estimates hop count from the HopLimitInd tag, and records delivery ratio and the longest reception gap. AcceptedHopCanvasVisualizer draws selected accepted one-hop frames; it does not prove that every drawn packet reached the destination. Summary scalars are written when finish() runs.

Recording guide. scalar-recording is enabled for summary values. vector-recording is disabled globally to keep files small, then enabled for PingApp RTT, metrics.destinationDelay, and metrics.destinationHopCount. Enable additional vectors only when a question needs a time history, because vector files grow with packet rate and run length.

5 Observe the route change

Simulation time	What to inspect
0-2 s AODV / 0-10 s DSDV	AODV discovers on demand; DSDV uses this quiet period for proactive table convergence.
18-24 s	The relay moves from (520,325) to (520,500). Watch the central route break and protocol-specific recovery.
24-36 s	The relay remains away. Check whether the upper bypass carries data and how hop count changes.
36-42 s	The relay returns. Observe AODV rediscovery or DSDV reconvergence.
After recovery	Finish the run and compare scalars, selected vectors, and event-log messages.

The link may fail or recover partway through movement; the 250 m threshold and AODV timers determine the exact time. RREQ floods seek a route, RREP returns a usable path, and RERR reports a broken active route. Black depicts ping traffic and gold traces relay movement. A red DROP label marks selected packet failures, not routine wireless overhearing. Animation speed changes viewing time only.

6 Find and analyze results

1. After the run finishes, refresh Project Explorer and open the results folder. OMNeT++ writes .sca scalars and selected .vec vectors; an .elog event log is also recorded.
2. Filter on metrics and host[0].app[0]. PingApp. Compare sent, received, lost, delivery ratio, destination delay, destination hops, RTT, and longest reception gap.
3. Plot metrics.destinationHopCount and metrics.destinationDelay against simulation time. Plot RTT beside them. A missing sample during a gap means no request reached the destination, not zero delay.
4. Repeat with a different range, activeRouteTimeout, or helloInterval. Record the complete parameter set because a change can affect both delivery and control overhead. For protocol comparison, match traffic start, run length, range, recording settings, and seeds.

Recorded result	How to interpret it
PingApp sent/received/lost	Offered requests and end-to-end reply outcomes; replies can fail independently of requests.
destination PDR	Destination deliveries divided by request attempts. Record the traffic window beside it.
destination delay and hops	One-way delay and estimated links for delivered requests only.
RTT vector	Round-trip time measured by PingApp; compare spikes with route changes.
longest reception gap	Largest interval without a destination request arrival during the run.
protocol parameters	Range, timers, traffic settings, run length, seed, and recording flags define the experiment.
AODV messages	RREQ, RREP, and RERR arrows show on-demand discovery and repair.
DSDV updates	Orange selected advertisements show proactive information propagation; every node still updates periodically.

Use destination delivery and PingApp replies together. A request can reach host[9] while its reply fails on the return path, so the two counts can differ. The longest gap identifies disruption severity but not its cause; use event-log times and route messages to explain it.

7 Student analysis questions

- Identify the first successful path and record its hop count. Which RREQ and RREP events precede the first delivered ping?
- Around relay departure, record the last pre-break arrival and first later arrival. Compare the difference with the longest reception gap.
- Change activeRouteTimeout and helloInterval separately. Predict how quickly AODV detects stale routes and how control traffic changes.
- Repeat with a smaller communicationRange or a shorter sendInterval. Explain the expected effect on route discoveries, drops, delay, and vector size.
- Report sent, destination-received, source replies, destination PDR, mean one-way delay, mean RTT, longest gap, and all changed parameters.

8 Troubleshooting and limits

- Unknown NED type or missing inet header: confirm that inet is open, built, referenced, and visible in the NED and C++ build paths.
- Custom module not found: confirm all custom C++ files were compiled and the matching NED files are on the NED path.
- Only a generic network appears: select AodvMobilityDemo. The base network does not show the AODV-specific legend.
- No metrics scalars: let the 100 s simulation finish normally, then refresh the results directory.
- No colored arrows: use QtEnv, slow the animation, and verify that the visualizer packet filters are enabled.
- This is an instructional wireless MANET experiment. Quantitative comparisons require aligned INET versions, traffic windows, radio parameters, and multiple seeds.

Reference documentation

OMNeT++ User Guide: <https://doc.omnetpp.org/omnetpp/UserGuide.pdf>

INET installation: <https://inet.omnetpp.org/Installation.html>

INET usage and project references: <https://inet.omnetpp.org/docs/users-guide/ch-usage.html>